

Lab Experiment – 5

Anshul(241210020)

Aim:

To study various instruction addressing modes by executing and analyzing a C program through GNU Debugger (GDB), and to observe assembly instructions, memory references, and register operations.

Theory:

1. Introduction to Addressing Modes

Addressing modes define the method by which an instruction accesses its operands. Different addressing techniques allow processors to efficiently retrieve data from registers, constants, memory locations, or stack frames.

Using GDB commands such as `info registers` and `disassemble`, we can inspect machine-level execution and understand how high-level C code is translated into assembly instructions.

2. Immediate Addressing Mode

In Immediate Addressing Mode, constant values are directly embedded within the instruction.

Examples:

- `mov $0xa, ...` → loads value 10 directly
- `mov $0x14, ...` → loads value 20 directly
- `movl $0x1, ...` → stores constant value

Observation:

This mode is mainly used for initializing variables or assigning fixed values.

3. Register Addressing Mode

In this mode, operands are stored in CPU registers, enabling faster execution.

Examples:

- `mov %rsp, %rbp`
- `mov %eax, %esi`
- `add %edx, %eax`
- `xor %eax, %eax`

Observation:

Register operations are fast because data is processed directly inside CPU registers.

4. Direct (Absolute) Addressing Mode

This mode directly references specific memory addresses or function locations.

Examples:

- `callq printf@plt`
- `callq __stack_chk_fail@plt`

Observation:

Used for function calls and accessing predefined memory locations.

5. Register Indirect Addressing Mode

In this mode, a register contains the memory address of the operand.

Examples:

- `mov (%rax), %eax`
- `mov %rax, ...`
- `lea -0x34(%rbp), %rax`

Observation:

Frequently used in pointer operations and array handling.

6. Indexed/Base Addressing Mode

The effective memory address is calculated using a base register and an offset.

Examples:

- `mov -0x34(%rbp), %edx`
- `mov -0x30(%rbp), %eax`
- `lea -0x34(%rbp), %rax`

Observation:

Useful for local variables, arrays, and structured data storage.

7. Relative Addressing Mode

Relative addressing determines target addresses relative to the current instruction pointer.

Examples:

- `je` (jump if equal)
- `jmp` (unconditional jump)

Observation:

Primarily used for loops, branching, and conditional statements.

8. Stack Addressing Mode

Stack operations manage function calls, local variables, and return addresses.

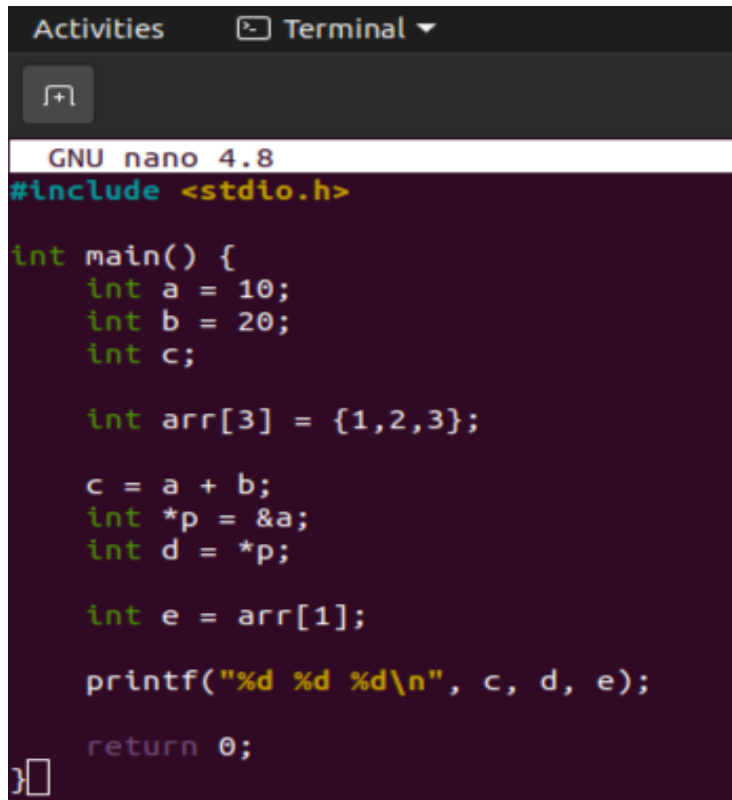
Examples:

- `push %rbp`
- `mov %rsp, %rbp`
- `sub $0x40, %rsp`
- `leaveq`

Observation:

This mode manages stack frames during program execution.

Program Used:

A screenshot of a Linux terminal window. The title bar shows 'Activities' and 'Terminal'. The terminal content shows the GNU nano 4.8 editor with a C program. The code includes <stdio.h>, defines a main function with variables a, b, c, an array arr, a pointer p, and a variable d. It calculates c = a + b, sets p to the address of a, sets d to the value at p, sets e to arr[1], and prints the values of c, d, and e. The program returns 0.

```
GNU nano 4.8
#include <stdio.h>

int main() {
    int a = 10;
    int b = 20;
    int c;

    int arr[3] = {1,2,3};

    c = a + b;
    int *p = &a;
    int d = *p;

    int e = arr[1];

    printf("%d %d %d\n", c, d, e);

    return 0;
}
```

GDB Commands Used:

- `break main` → Set breakpoint
- `run` → Start execution
- `step` → Execute line-by-line
- `continue` → Resume execution
- `info registers` → Display register contents
- `disassemble main` → View assembly instructions

Code output :-

```
(gdb) break main
Breakpoint 1 at 0x1169: file asst.c, line 3.
(gdb) run
Starting program: /home/nitdelhi/anshul/ass
```

```
Breakpoint 1, main () at asst.c:3
```

```
3      int main() {
(gdb) step
4      int a = 10;
(gdb) step
5      int b = 20;
(gdb) step
8      int arr[3] = {1,2,3};
(gdb) step
10     c = a + b;
(gdb) step
11     int *p = &a;
(gdb) step
12     int d = *p;
```

```
(gdb) continue
```

```
Continuing.
```

```
30 10 2
```

```
[Inferior 1 (process 6619) exited normally]
```

```
(gdb) disassemble main
```

```
Dump of assembler code for function main:
```

```
0x000055555555169 <+0>:      endbr64
0x00005555555516d <+4>:      push    %rbp
0x00005555555516e <+5>:      mov     %rsp,%rbp
0x000055555555171 <+8>:      sub     $0x40,%rsp
0x000055555555175 <+12>:     mov     %fs:0x28,%rax
0x00005555555517e <+21>:     mov     %rax,-0x8(%rbp)
0x000055555555182 <+25>:     xor     %eax,%eax
0x000055555555184 <+27>:     movl    $0xa,-0x34(%rbp)
0x00005555555518b <+34>:     movl    $0x14,-0x30(%rbp)
```

```
0x000055555555192 <+41>:     movl    $0x1,-0x14(%rbp)
0x000055555555199 <+48>:     movl    $0x2,-0x10(%rbp)
0x0000555555551a0 <+55>:     movl    $0x3,-0xc(%rbp)
0x0000555555551a7 <+62>:     mov     -0x34(%rbp),%edx
0x0000555555551aa <+65>:     mov     -0x30(%rbp),%eax
0x0000555555551ad <+68>:     add     %edx,%eax
0x0000555555551af <+70>:     mov     %eax,-0x2c(%rbp)
0x0000555555551b2 <+73>:     lea     -0x34(%rbp),%rax
0x0000555555551b6 <+77>:     mov     %rax,-0x20(%rbp)
0x0000555555551ba <+81>:     mov     -0x20(%rbp),%rax
0x0000555555551be <+85>:     mov     (%rax),%eax
0x0000555555551c0 <+87>:     mov     %eax,-0x28(%rbp)
0x0000555555551c3 <+90>:     mov     -0x10(%rbp),%eax
0x0000555555551c6 <+93>:     mov     %eax,-0x24(%rbp)
0x0000555555551c9 <+96>:     mov     -0x24(%rbp),%ecx
0x0000555555551cc <+99>:     mov     -0x28(%rbp),%edx
0x0000555555551cf <+102>:    mov     -0x2c(%rbp),%eax
0x0000555555551d2 <+105>:    mov     %eax,%esi
0x0000555555551d4 <+107>:    lea     0xe29(%rip),%rdi      # 0x555555556004
0x0000555555551db <+114>:    mov     $0x0,%eax
0x0000555555551e0 <+119>:    callq   0x55555555070 <printf@plt>
0x0000555555551e5 <+124>:    mov     $0x0,%eax
0x0000555555551ea <+129>:    mov     -0x8(%rbp),%rsi
0x0000555555551ee <+133>:    xor     %fs:0x28,%rsi
0x0000555555551f7 <+142>:    je      0x555555551fe <main+149>
0x0000555555551f9 <+144>:    callq   0x55555555060 <__stack_chk_fail@plt>
0x0000555555551fe <+149>:    leaveq
0x0000555555551ff <+150>:    retq
```

```
End of assembler dump.
```

```
(gdb) q
```

```
nitdelhi@master:~/anshul$
```